

Line-by-Line Explanation of Azure DevOps Terraform YAML Pipeline

This document provides a detailed line-by-line explanation of an Azure DevOps YAML pipeline used to automate Terraform deployments to Microsoft Azure. The pipeline installs Terraform, authenticates with Azure, initializes Terraform, validates the configuration, generates an execution plan, and applies the infrastructure changes.

1. Trigger

trigger:

- main

The trigger keyword specifies when the pipeline should start automatically.

'- main' means the pipeline runs whenever code is pushed to the main branch.

2. Agent Pool

pool:

- vmImage: 'ubuntu-latest'

The pool section specifies the build agent.

'ubuntu-latest' creates a temporary Ubuntu virtual machine that executes the pipeline tasks.

3. Variables

variables:

- azureServiceConnection: 'Permanent Connection'

Variables store reusable values.

The Azure Service Connection contains Azure authentication details and allows the pipeline to log into Azure securely.

4. Install Terraform Task

- task: TerraformInstaller@1

displayName: Install Terraform

inputs:

- terraformVersion: '1.5.7'

This task downloads and installs Terraform version 1.5.7 on the build agent.

5. Terraform Init Task

- task: AzureCLI@2

Uses Azure CLI to authenticate with Azure.

displayName: Terraform Init

Shows a readable task name in pipeline logs.

azureSubscription:

Uses the Azure Service Connection.

scriptType: bash

Runs Bash commands.

scriptLocation: inlineScript

The script is written directly in YAML.

Commands:

az --version

Displays Azure CLI version.

terraform --version

Displays Terraform version.

terraform init

Initializes Terraform by downloading providers, modules, and creating the .terraform directory.

6. Terraform Validate Task

terraform validate

Checks Terraform configuration for syntax and logical errors.

Does not create Azure resources.

Returns 'Success! The configuration is valid.' if everything is correct.

7. Terraform Plan Task

terraform plan -var-file="terraform.tfvars"

Compares existing Azure infrastructure with the Terraform configuration.

Shows resources to be created, modified, or deleted.

Uses values stored in terraform.tfvars.

8. Terraform Apply Task

terraform apply -auto-approve -var-file="terraform.tfvars"

Creates or updates Azure resources.

-auto-approve:

Skips manual confirmation.

-var-file:

Loads configuration values from terraform.tfvars.

9. Pipeline Execution Flow

Code pushed to main

↓

Pipeline starts

↓

Create Ubuntu build agent

↓

Install Terraform

↓

Login to Azure

↓

Terraform Init

↓

Terraform Validate

↓

Terraform Plan

↓

Terraform Apply

↓

Deploy Azure infrastructure

10. Best Practices

1. Always run init before validate, plan, and apply.
2. Store variable values in terraform.tfvars.
3. Separate Plan and Apply stages in production.
4. Add manual approval before Apply for production deployments.
5. Use remote state storage for team environments.

Conclusion

This Azure DevOps YAML pipeline follows the standard Terraform workflow: terraform init, terraform validate, terraform plan, and terraform apply. It automates infrastructure deployment to Azure while maintaining a structured and repeatable CI/CD process.